

RL hw4

110612025 魏于翔

June 23 2026

1 Problem 1

1.1 (a)

The exact gradient of the DPO loss function ($\nabla_{\theta} \mathcal{L}_{DPO}$) is:

$$\begin{aligned} \nabla_{\theta} \mathcal{L}_{DPO}(\pi_{\theta}; \pi_{ref}) = & \\ -\beta E_{(x, y_w, y_l) \sim \mathcal{D}} & \left[\sigma(\hat{r}_{\theta}(x, y_l) - \hat{r}_{\theta}(x, y_w)) \right. \\ & \cdot [\nabla_{\theta} \log \pi(y_w|x) - \nabla_{\theta} \log \pi(y_l|x)] \end{aligned} \quad (1)$$

Each part intuition behind the gradient:

- $\sigma(\hat{r}_{\theta}(x, y_l) - \hat{r}_{\theta}(x, y_w))$: When the model is correct, means $r_w > r_l$, so the scaling weight $w = \sigma(r_l - r_w) \rightarrow 0$, meaning that the model barely change, and if the model doesn't sure which is better, it give them medium weight. As for the model is wrong, meaning that $r_w < r_l$, it will give the model's weight $w = \sigma(r_l - r_w) \rightarrow 1$ to change the model. In conclusion, this term means that higher weight when reward estimate is wrong.
- $\nabla_{\theta} \log \pi(y_w|x)$: This term is the winner's policy gradient. In the policy gradient formula: $\theta \leftarrow \theta - \eta \Delta L$. We can see in the ΔL 's formula, there is a negative sign outside, so the update becomes, $+\log \pi(y_w|x)$, adding the winner's probability.
- $-\nabla_{\theta} \log \pi(y_l|x)$: This term is the loser's policy gradient. In the policy gradient formula: $\theta \leftarrow \theta - \eta \Delta L$. We can see in the ΔL 's formula, there is a negative sign outside, so the update becomes, $-\log \pi(y_l|x)$, decreasing the loser's probability.

1.2 (b)

- Question 1: In **DataCollatorForPreference** class's function **torch_call()**, it divide the prompt and the answer into prompt_chosen_ids and prompt_rejected_ids, also build the completion_mask, 0 for prompt token and 1 for response token. And then build the input_ids composed by B chosen and B rejected.

In **__compute_loss()** function, pass the model's parameters in and get the *output.logits* and use the token_t to predict the token_(t+1) and compute each token's log-probability in per_token_logps variable, so each position has it's $\log \pi_{\theta}(y_t|y_{<t}, x)$. And then we imply the shift_completion_mask to clean up the prompt token's logprob, and then add it up to become the chosen_logps or rejected_logps.

Because we design the input as B's chosen prompt and B's rejected prompt, so we just cut in the half to get the corresponding results.

- Question 2: You can choose to cut the beginning or the end of the sequence by setting the **truncation_mode** variable, if you want to truncate the end, set the variable to keep_start mode, if you want to truncate the beginning, set the variable to keep_end mode.
- Question 3: The standard DPO loss that involves $\log \sigma(x)$, this term may cause the numerical stability issue, when x is the negative number like -1000, the $\sigma(x) \rightarrow 0$, and when you want to get $\log(0) \rightarrow -\inf$, which may cause the trouble. In DPOTrainer, it handle the problem by using **logsigmoid** function in pytorch, in the function it using the mathematical identity

$\log \sigma(x) = -\log(1 + e^{-x})$ to avoid the problem we mentioned before.

- **Question 4:** In the original DPO loss, $h = (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x))$, and $L = -\log \sigma(\beta h)$. However when we choose the loss type with **ipo**, TRL replace this classification-style with the IPO squared loss. First step is to convert sequence-level log-probabilities into average log-probability per completion token, because IPO's squared loss is sensitive to response length. Without normalization, longer responses would naturally produce larger log-probability differences and therefore disproportionately larger losses. Then use the above number to calculate the average token log-probabilities. Last step is to apply the IPO objective by using $L_{IPO} = (\Delta_{IPO} - \frac{1}{2\beta})^2$. Instead of maximizing the preference margin indefinitely, IPO trains the model to match a target margin $\frac{1}{2\beta}$.

[illegible]

Figure 1:

- Question 5: TRL build a frozen reference policy π_{ref} . In normal case: we copy the policy and froze it so it can't change. In PEFT case: Shut down the adapter, use base model as reference.

```

[Header: 4.8]
First eval Dataset: ['chosen', 'I': ['content': 'As an HR manager, you want to test a potential employee's ability to solve puzzles
to determine their suitability for a job. Write a Python script that generates a list of questions that require logical reasoning
and problem-solving skills. The questions should be categorized into five groups: 'Mathematical puzzles', 'Language puzzles', 'Logic
puzzles', 'Word puzzles', and 'Pattern recognition puzzles'. Use the following code as a starting point/questions = [
    "Mathematical puzzles": ["If the value of x+y = 20 and x-y = 10, what is the value of x and y?", "If a pizza has a radius of 8 inches and is
cut into 4 equal slices, what is the area of each slice?"], "Language puzzles": ["What word starts with 'A' and ends with 'E' and
contains only vowels?", "What word is a synonym for 'happy' but contains only consonants?"], "Logic puzzles": ["You have 3 boxes. One contains only apples, one
contains only oranges, and one contains both apples and oranges. The boxes have been incorrectly labeled such that no label is
correct. You are allowed to open one box and take out one fruit. Based on this information, how can you correctly label all three
boxes?", "Pattern recognition puzzles": ["What is the next number in the sequence: 1, 3, 4, 6, 10, 15, 21, 28, 36, 45, 55, 66, 78, 91, 105, 120, 136, 153, 171, 190, 210, 231, 252, 273, 294, 315, 336, 357, 378, 400, 420, 441, 462, 483, 504, 525, 546, 567, 588, 609, 630, 651, 672, 693, 714, 735, 756, 777, 798, 819, 840, 861, 882, 903, 924, 945, 966, 987, 1008, 1029, 1050, 1071, 1092, 1113, 1134, 1155, 1176, 1197, 1218, 1239, 1260, 1281, 1302, 1323, 1344, 1365, 1386, 1407, 1428, 1449, 1470, 1491, 1512, 1533, 1554, 1575, 1596, 1617, 1638, 1659, 1680, 1701, 1722, 1743, 1764, 1785, 1806, 1827, 1848, 1869, 1890, 1911, 1932, 1953, 1974, 1995, 2016, 2037, 2058, 2079, 2100, 2121, 2142, 2163, 2184, 2205, 2226, 2247, 2268, 2289, 2310, 2331, 2352, 2373, 2394, 2415, 2436, 2457, 2478, 2499, 2520, 2541, 2562, 2583, 2604, 2625, 2646, 2667, 2688, 2709, 2730, 2751, 2772, 2793, 2814, 2835, 2856, 2877, 2898, 2919, 2940, 2961, 2982, 3003, 3024, 3045, 3066, 3087, 3108, 3129, 3150, 3171, 3192, 3213, 3234, 3255, 3276, 3297, 3318, 3339, 3360, 3381, 3402, 3423, 3444, 3465, 3486, 3507, 3528, 3549, 3570, 3591, 3612, 3633, 3654, 3675, 3696, 3717, 3738, 3759, 3780, 3801, 3822, 3843, 3864, 3885, 3906, 3927, 3948, 3969, 3990, 4011, 4032, 4053, 4074, 4095, 4116, 4137, 4158, 4179, 4200, 4221, 4242, 4263, 4284, 4305, 4326, 4347, 4368, 4389, 4410, 4431, 4452, 4473, 4494, 4515, 4536, 4557, 4578, 4599, 4620, 4641, 4662, 4683, 4704, 4725, 4746, 4767, 4788, 4809, 4830, 4851, 4872, 4893, 4914, 4935, 4956, 4977, 4998, 5019, 5040, 5061, 5082, 5103, 5124, 5145, 5166, 5187, 5208, 5229, 5250, 5271, 5292, 5313, 5334, 5355, 5376, 5397, 5418, 5439, 5460, 5481, 5502, 5523, 5544, 5565, 5586, 5607, 5628, 5649, 5670, 5691, 5712, 5733, 5754, 5775, 5796, 5817, 5838, 5859, 5880, 5901, 5922, 5943, 5964, 5985, 6006, 6027, 6048, 6069, 6090, 6111, 6132, 6153, 6174, 6195, 6216, 6237, 6258, 6279, 6300, 6321, 6342, 6363, 6384, 6405, 6426, 6447, 6468, 6489, 6510, 6531, 6552, 6573, 6594, 6615, 6636, 6657, 6678, 6699, 6720, 6741, 6762, 6783, 6804, 6825, 6846, 6867, 6888, 6909, 6930, 6951, 6972, 6993, 7014, 7035, 7056, 7077, 7098, 7119, 7140, 7161, 7182, 7203, 7224, 7245, 7266, 7287, 7308, 7329, 7350, 7371, 7392, 7413, 7434, 7455, 7476, 7497, 7518, 7539, 7560, 7581, 7602, 7623, 7644, 7665, 7686, 7707, 7728, 7749, 7770, 7791, 7812, 7833, 7854, 7875, 7896, 7917, 7938, 7959, 7980, 8001, 8022, 8043, 8064, 8085, 8106, 8127, 8148, 8169, 8190, 8211, 8232, 8253, 8274, 8295, 8316, 8337, 8358, 8379, 8400, 8421, 8442, 8463, 8484, 8505, 8526, 8547, 8568, 8589, 8610, 8631, 8652, 8673, 8694, 8715, 8736, 8757, 8778, 8799, 8820, 8841, 8862, 8883, 8904, 8925, 8946, 8967, 8988, 9009, 9030, 9051, 9072, 9093, 9114, 9135, 9156, 9177, 9198, 9219, 9240, 9261, 9282, 9303, 9324, 9345, 9366, 9387, 9408, 9429, 9450, 9471, 9492, 9513, 9534, 9555, 9576, 9597, 9618, 9639, 9660, 9681, 9702, 9723, 9744, 9765, 9786, 9807, 9828, 9849, 9870, 9891, 9912, 9933, 9954, 9975, 9996, 10017, 10038, 10059, 10080, 10101, 10122, 10143, 10164, 10185, 10206, 10227, 10248, 10269, 10290, 10311, 10332, 10353, 10374, 10395, 10416, 10437, 10458, 10479, 10500, 10521, 10542, 10563, 10584, 10605, 10626, 10647, 10668, 10689, 10710, 10731, 10752, 10773, 10794, 10815, 10836, 10857, 10878, 10899, 10920, 10941, 10962, 10983, 11004, 11025, 11046, 11067, 11088, 11109, 11130, 11151, 11172, 11193, 11214, 11235, 11256, 11277, 11298, 11319, 11340, 11361, 11382, 11403, 11424, 11445, 11466, 11487, 11508, 11529, 11550, 11571, 11592, 11613, 11634, 11655, 11676, 11697, 11718, 11739, 11760, 11781, 11802, 11823, 11844, 11865, 11886, 11907, 11928, 11949, 11970, 11991, 12012, 12033, 12054, 12075, 12096, 12117, 12138, 12159, 12180, 12201, 12222, 12243, 12264, 12285, 12306, 12327, 12348, 12369, 12390, 12411, 12432, 12453, 12474, 12495, 12516, 12537, 12558, 12579, 12600, 12621, 12642, 12663, 12684, 12705, 12726, 12747, 12768, 12789, 12810
```

2 Problem 2

(a)

Figure 2:

